
ECUAȚIA DE GRADUL AL II-LEA

PROIECT TESTAREA SISTEMELOR SOFTWARE
T1 TESTARE UNITARĂ ÎN PYTHON

BILIUȚĂ Elisa-Adelina ¹¹, MARCOVICI Anamaria ²¹ și
NEDELESCU George-Eduard ³¹

¹Universitatea din București/ Facultatea de Matematică și Informatică/
Programul de studii universitare de licență Matematică-Informatică

Anul III, Seria 31, Grupa 311

¹elisa-adelina.biliuta@s.unibuc.ro

²anamaria.marcovici@s.unibuc.ro

³george-eduard.nedelescu@s.unibuc.ro

Cuprins

1	Prezentarea problemei	2
2	Testare funcțională	4
2.1	Partiționarea în clase de echivalență (CE)	4
2.2	Analiza valorilor de frontieră (AVF)	7
2.3	Partiționarea în categorii (PC)	9
2.3.1	Graful cauză-efect	12
3	Testare structurală	16
3.1	Graful de flux de control (CFG)	16
3.1.1	Generarea automată a CFG-ului	16
3.2	Analiza acoperirii (Coverage)	19
3.2.1	Acoperire la nivel de instrucțiuni	19
3.2.2	Legătura dintre cazurile de test și acoperirea codului	20
3.3	Rularea instrumentului coverage.py	20
3.4	Acoperire la nivel de condiție	21
3.4.1	Condition/Decision Coverage	22
3.4.2	Multiple Condition Coverage	22
3.4.3	MC/DC	22
3.4.4	Path Coverage	24
3.4.5	LCSAJ	24
3.5	Complexitatea ciclomatică (McCabe)	24
3.6	Circuite independente	25

Capitolul 1

Prezentarea problemei

În acest capitol prezentăm problema principală pe care o vom supune testării, anume rezolvarea unei ecuații de gradul al II-lea de forma $ax^2 + bx + c = 0$, cu $a \in \mathbb{R}^*$ și $b, c \in \mathbb{R}$.

Programul implementat primește trei coeficienți reali (a , b , și c) și calculează soluțiile ecuației pe baza discriminantului $\Delta = b^2 - 4ac$. De asemenea, programul tratează cazurile de excepție, cum ar fi validarea tipului de date introdus și restricția matematică pentru care coeficientul $a \neq 0$.

Distingem mai jos cele trei cazuri posibile, precum și formulele de calcul ale soluțiilor ecuației, în funcție de semnul lui Δ :

$$x_{1,2} = \begin{cases} \frac{-b \pm \sqrt{\Delta}}{2a}, & \text{dacă } \Delta > 0 \\ -\frac{b}{2a}, & \text{dacă } \Delta = 0 \\ \frac{-b \pm i\sqrt{|\Delta|}}{2a}, & \text{dacă } \Delta < 0 \end{cases}.$$

În continuare, prezentăm implementarea clasei `EquationSolver` în limbajul Python.

```
1 import math
2
3 class EquationSolver:
4     def __init__(self, a, b, c):
5         self.a = a
6         self.b = b
7         self.c = c
8
9     def solve(self):
10        if not all(isinstance(x, (int, float)) and not isinstance(x, bool)
11        for x in [self.a, self.b, self.c]):
12            raise ValueError("Coeficientii ecuatiei trebuie sa fie numere
13            reale!")
14
15        if self.a == 0:
16            raise ValueError("Coeficientul a nu poate fi 0!")
17
18        delta = self.b ** 2 - 4 * self.a * self.c
19
20        if math.isclose(delta, 0.0, abs_tol=1e-9):
21            return -self.b / (2 * self.a)
22
23        elif delta < 0:
24            real_part = -self.b / (2 * self.a)
25            imag_part = math.sqrt(abs(delta)) / (2 * self.a)
26            x1 = complex(real_part, imag_part)
27            x2 = complex(real_part, -imag_part)
28            return x1, x2
29
30        else:
31            x1 = (-self.b + math.sqrt(delta)) / (2 * self.a)
32            x2 = (-self.b - math.sqrt(delta)) / (2 * self.a)
33            return x1, x2
```

Secvență de cod 1.1: Fișierul equation_solver.py ilustrând clasa EquationSolver

Capitolul 2

Testare funcțională

În acest capitol detaliem strategiile de testare funcțională de tip *Black Box* aplicate asupra clasei `EquationSolver`. Testarea funcțională presupune generarea datelor de test pe baza specificațiilor programului, fără a ține cont de structura internă a codului.

2.1 Partiționarea în clase de echivalență (CE)

Domeniul de intrări este format din cei trei coeficienți (a, b, c) , care trebuie să fie numere reale, iar $a \neq 0$. Pe baza verificării tipului de date introduse, a restricțiilor și a calculului discriminantului (Δ) , am identificat un domeniu de ieșiri format din 5 tipuri distincte de rezultate. În continuare, vom analiza fiecare dintre aceste domenii și clasele de echivalență generate.

Domeniul de intrări conține:

- un număr real nenul a : determină clasele $A_1 = \mathbb{R}^*$, $A_2 = \{0\}$ și $A_3 = \mathbb{R}^c$;
- un număr real b : determină clasele $B_1 = \mathbb{R}$ și $B_2 = \mathbb{R}^c$;
- un număr real c : determină clasele $C_1 = \mathbb{R}$ și $C_2 = \mathbb{R}^c$.

Observație 1. Am notat cu $\mathbb{R}^c$ complementara mulțimii numerelor reale. Dacă $x \in \mathbb{R}^c$, atunci x are tipul de dată diferit de `int` și `float` în reprezentarea internă a limbajului Python (și nu are nici tip `bool`), lucru ce va genera erori la validarea tipului de date.

Domeniul de ieșiri constă în rezultatele posibile returnate de metoda `solve()` și poate fi explicitat astfel:

- mulțimea soluțiilor ecuației: determină clasele $S_1 = \mathbb{C} \times \mathbb{C}$, $S_2 = \mathbb{R}$, $S_3 = \mathbb{R} \times \mathbb{R}$;
- stările de eroare: determină clasele: $E_1 = \text{ValueError}$ (excepție aruncată când cel puțin un coeficient aparține mulțimii $\mathbb{R}^c$) și $E_2 = \text{ValueError}$ (excepție aruncată pentru restricția matematică $a \neq 0$).

Observație 2. Clasa S_1 presupune un tuplu de două rădăcini complexe conjugate (când $\Delta < 0$), clasa S_2 presupune o singură rădăcină reală (rădăcină dublă, când $\Delta = 0$), iar clasa S_3 presupune un tuplu de două rădăcini reale distincte (când $\Delta > 0$).

Pe baza domeniului de ieșiri, folosind principiul metodei „reverse engineering” (de la ieșiri spre intrări) și ținând cont de fluxul de validare care previne evaluările multiple (o excepție aruncată oprește execuția), putem partiționa domeniul de intrări în trei clase disjuncte:

1. **Intrări care declanșează eroarea E_1 (Validarea tipului):** Execuția se oprește prematur dacă cel puțin un coeficient nu este număr real. Aceasta generează prima clasă de echivalență globală:

$$C_{E_1} = \{(a, b, c) \mid a \in A_3 \vee b \in B_2 \vee c \in C_2\}$$

2. **Intrări care nu declanșează eroarea E_1 , dar care declanșează eroarea E_2 (Restricția matematică):** Acest caz presupune că toate datele sunt numere reale ($a \in A_1 \cup A_2$, $b \in B_1$, $c \in C_1$), dar coeficientul dominant este zero ($a \in A_2$). Obținem astfel a doua clasă de echivalență globală:

$$C_{E_2} = A_2 \times B_1 \times C_1 = \{(0, b, c) \mid b, c \in \mathbb{R}\}$$

3. **Intrări care nu declanșează nicio eroare:** Acest caz presupune date valide: avem coeficienți reali și $a \neq 0$, adică domeniul $A_1 \times B_1 \times C_1$. Facem discuție după calculul discriminantului Δ , care subpartiționează domeniul valid în cele 3 clase corespunzătoare soluțiilor:

$$C_{S_1} = \{(a, b, c) \in A_1 \times B_1 \times C_1 \mid b^2 - 4ac < 0\}$$

$$C_{S_2} = \{(a, b, c) \in A_1 \times B_1 \times C_1 \mid b^2 - 4ac = 0\}$$

$$C_{S_3} = \{(a, b, c) \in A_1 \times B_1 \times C_1 \mid b^2 - 4ac > 0\}$$

Pe baza partiționării anterioare, am selectat câte un caz de testare reprezentativ pentru fiecare clasă de echivalență. Seturile de date și rezultatele așteptate sunt sintetizate în tabelul de mai jos:

ID Test	Clasa	a	b	c	Δ	Rezultat așteptat
TC_CE_01	C_{E_1}	"text"	2	1	N/A	E_1 (Excepție: tipuri invalide)
TC_CE_02	C_{E_2}	0	2	1	N/A	E_2 (Excepție: a nu poate fi 0)
TC_CE_03	C_{S_1}	1	1	1	-3	S_1 (Două rădăcini complexe)
TC_CE_04	C_{S_2}	1	2	1	0	S_2 (Rădăcină reală dublă)
TC_CE_05	C_{S_3}	1	5	6	1	S_3 (Două rădăcini reale distincte)

Tabela 2.1: Cazuri de testare pentru metoda CE

Observație 3. Se remarcă faptul că pentru clasele de excepție C_{E_1} și C_{E_2} , discriminantul Δ nu este calculat, deoarece execuția este întreruptă înainte de a ajunge la evaluarea formulei matematice. Această afirmație nu încalcă principiile strategiei *Black Box*.

Pentru a valida practic scenariile teoretice definite anterior, în secvența de cod următoare prezentăm implementarea automatizată a cazurilor de testare extrase din Tabelul 2.1, utilizând frameworkul `unittest` din limbajul Python. Deoarece urmează se efectuează calcule analitice pe valori reale, verificările folosesc extensiv mecanismul `assertAlmostEqual` pentru a preveni eșuarea testelor pe fondul erorilor inevitabile de trunchiere și rotunjire în virgulă mobilă.

```
1 class TestEquationSolverCE(unittest.TestCase):
2     # TC_CE_01: Clasa CE1 - Validarea tipului de date (cel puțin un
    # coeficient nu este număr real); vom trata fenomenul defect masking
3     def test_tc_ce_01_a_invalid(self):
4         solver = EquationSolver("text", 2, 1)
5         with self.assertRaises(ValueError):
6             solver.solve()
7
8     def test_tc_ce_01_b_invalid(self):
9         solver = EquationSolver(1, "text", 1)
10        with self.assertRaises(ValueError):
11            solver.solve()
12
13    def test_tc_ce_01_c_invalid(self):
14        solver = EquationSolver(1, 2, "text")
15        with self.assertRaises(ValueError):
16            solver.solve()
17
18    # TC_CE_02: Clasa CE2 - Restricția matematică ( $a \neq 0$ )
19    def test_tc_ce_02_a_zero(self):
20        solver = EquationSolver(0, 2, 1)
21        with self.assertRaises(ValueError):
22            solver.solve()
23
24    # TC_CE_03: Clasa CS1 -  $\Delta < 0$  (Două rădăcini complexe conjugate)
25    def test_tc_ce_03_delta_negativ(self):
26        solver = EquationSolver(1, 1, 1)
27        x1, x2 = solver.solve()
28
29        self.assertIsInstance(x1, complex)
30        self.assertIsInstance(x2, complex)
31
32        self.assertAlmostEqual(x1.real, -0.5)
33        self.assertAlmostEqual(x2.real, -0.5)
34
35        self.assertAlmostEqual(max(x1.imag, x2.imag), 0.866025, places=5)
36        self.assertAlmostEqual(min(x1.imag, x2.imag), -0.866025, places=5)
37
38    # TC_CE_04: Clasa CS2 -  $\Delta = 0$  (0 rădăcină reală dublă)
39    def test_tc_ce_04_delta_zero(self):
40        solver = EquationSolver(1, 2, 1)
41        x = solver.solve()
42
43        self.assertIsInstance(x, float)
44        self.assertAlmostEqual(x, -1.0)
45
46    # TC_CE_05: Clasa CS3 -  $\Delta > 0$  (Două rădăcini reale distincte)
47    def test_tc_ce_05_delta_pozitiv(self):
48        solver = EquationSolver(1, 5, 6)
49        x1, x2 = solver.solve()
50
51        self.assertIsInstance(x1, float)
52        self.assertIsInstance(x2, float)
53
54        self.assertAlmostEqual(max(x1, x2), -2.0, places=5)
55        self.assertAlmostEqual(min(x1, x2), -3.0, places=5)
```

Secvență de cod 2.1: Fișierul test_equation_solver.py ilustrând clasa TestEquationSolverCE

2.2 Analiza valorilor de frontieră (AVF)

Conform principiilor de bază ale metodei AVF, pentru fiecare frontieră identificată trebuie testată atât valoarea exactă a limitei, cât și valorile imediat adiacente (din interiorul și din afara clasei valide). Pentru problema ecuației de gradul al II-lea, deși prelucrăm un domeniu continuu, reprezentat de mulțime $\mathbb{R}$, distingem clar două frontiere decizionale impuse de fluxul de control al programului: trecerea dintre valid și invalid ($a = 0$) și punctul critic dintre rădăcinile reale și cele complexe ($\Delta = 0$). Să detaliem:

- **Frontiera coeficientului dominant (a):** Restricția matematică impune $a \neq 0$. Conform teoriei AVF, setul de teste trebuie să evalueze cazul-limită ($a = 0$) și valorile imediat adiacente acestuia ($a \rightarrow 0^+$ și $a \rightarrow 0^-$). Întrucât testul pentru valoarea $a = 0$ a fost deja executat sub umbrela metodei CE (reprezentând clasa de excepție C_{E_2}), analiza de frontieră va fi completată prin injectarea unor valori arbitrar de mici, notate cu $\pm\epsilon$ (unde vom alege $\epsilon = 0.001$).
- **Frontiera discriminantului (Δ):** Valoare critică care produce separarea rădăcinilor reale de cele complexe este $\Delta = 0$. Cazul-limită ($\Delta = 0$) reprezintă însăși clasa C_{S_2} din metoda CE. Tehnica AVF impune completarea cu testarea vecinătăților $\Delta \rightarrow 0^+$ și $\Delta \rightarrow 0^-$, stări care pot fi forțate prin variația fină a coeficientului b în jurul valorii 2, menținând $a = 1$ și $c = 1$, de exemplu.

Pentru a oferi o imagine completă a acoperirii la frontieră, Tabelul 2.2 sintetizează întregul spectru de teste AVF, înglobând atât valorile pentru frontieră (moștenite din partiționarea de echivalență), cât și cazurile pentru vecinătățile acesteia.

ID Test	Frontieră	a	b	c	Δ	Rezultat așteptat
TC_CE_02	$a = 0$	0	2	1	N/A	E_2 (Excepție: a nu poate fi 0)
TC_AVF_01	$a \rightarrow 0^-$	-0.001	2	1	4.004	S_3 (Două rădăcini reale distincte)
TC_AVF_02	$a \rightarrow 0^+$	0.001	2	1	3.996	S_3 (Două rădăcini reale distincte)
TC_CE_04	$\Delta = 0$	1	2	1	0	S_2 (Rădăcină reală dublă)
TC_AVF_03	$\Delta \rightarrow 0^-$	1	1.999	1	≈ -0.004	S_1 (Două rădăcini complexe)
TC_AVF_04	$\Delta \rightarrow 0^+$	1	2.001	1	≈ 0.004	S_3 (Două rădăcini reale distincte)

Tabela 2.2: Cazuri de testare pentru metoda AVF

Observație 4. Spre deosebire de mulțimile de tip `int`, unde valoarea adiacentă este precis determinabilă (+1 sau -1), pe un domeniu continuu de tip `float` conceptul de „valoare imediat adiacentă” este mărginit doar de limitele de reprezentare arhitecturală a mașinii. Din perspectiva strategiei *Black Box*, o toleranță mică $\epsilon = 10^{-3}$ este optimă și suficientă pentru a garanta evaluarea ramurilor decizionale la extremitățile lor critice.

Implementarea automatizată a scenariilor AVF este redată mai jos.


```

1 class TestEquationSolverAVF(unittest.TestCase):
2     # Testele pentru a=0 si Delta=0 au fost deja implementate in clasa
    TestEquationSolverCE (test_tc_ce_02 si test_tc_ce_04)
3
4     # TC_AVF_01: Limita stanga pentru coeficientul a (a -> 0-)
5     def test_tc_avf_01_a_limita_negativa(self):
6         solver = EquationSolver(-0.001, 2, 1)
7         x1, x2 = solver.solve()
8
9         self.assertIsInstance(x1, float)
10        self.assertIsInstance(x2, float)
11
12        self.assertAlmostEqual(max(x1, x2), 2000.49988, places=3)
13        self.assertAlmostEqual(min(x1, x2), -0.49987, places=3)
14
15    # TC_AVF_02: Limita dreapta pentru coeficientul a (a -> 0+)
16    def test_tc_avf_02_a_limita_pozitiva(self):
17        solver = EquationSolver(0.001, 2, 1)
18        x1, x2 = solver.solve()
19
20        self.assertIsInstance(x1, float)
21        self.assertIsInstance(x2, float)
22
23        self.assertAlmostEqual(max(x1, x2), -0.50012, places=3)
24        self.assertAlmostEqual(min(x1, x2), -1999.49987, places=3)
25
26    # TC_AVF_03: Limita stanga pentru Delta (Delta -> 0-)
27    def test_tc_avf_03_delta_limita_negativa(self):
28        solver = EquationSolver(1, 1.999, 1)
29        x1, x2 = solver.solve()
30
31        self.assertIsInstance(x1, complex)
32        self.assertIsInstance(x2, complex)
33
34        self.assertAlmostEqual(x1.real, -0.9995, places=3)
35        self.assertAlmostEqual(x2.real, -0.9995, places=3)
36
37        self.assertAlmostEqual(max(x1.imag, x2.imag), 0.03161, places=3)
38        self.assertAlmostEqual(min(x1.imag, x2.imag), -0.03161, places=3)
39
40    # TC_AVF_04: Limita dreapta pentru Delta (Delta -> 0+)
41    def test_tc_avf_04_delta_limita_pozitiva(self):
42        solver = EquationSolver(1, 2.001, 1)
43        x1, x2 = solver.solve()
44
45        self.assertIsInstance(x1, float)
46        self.assertIsInstance(x2, float)
47
48        self.assertAlmostEqual(max(x1, x2), -0.96887, places=3)
49        self.assertAlmostEqual(min(x1, x2), -1.03213, places=1)

```

Secvență de cod 2.2: Fișierul test_equation_solver.py ilustrând clasa TestEquationSolverAVF

2.3 Partiționarea în categorii (PC)

Metoda partiționării în categorii extinde tehnicile anterioare, având ca scop generarea unui set de date de test care acoperă exhaustiv funcționalitatea sistemului și maximizează posibilitatea descoperirii erorilor. Această abordare structurată descompune specificația în unități testabile, identifică parametrii, categoriile acestora și partiționează fiecare categorie în alternative. Pentru a preveni explozia combinatorică inerentă și a elimina cazurile care nu au sens, se introduc constrângeri logice (condiții de selecție) asupra alternativelor.

Urmărind pașii metodologici, aplicăm metoda PC pentru clasa `EquationSolver`:

Pasul 1: Descompunerea specificației în unități

Sistemul prezintă o singură unitate funcțională testabilă izolată: metoda `solve()`.

Pasul 2: Identificarea parametrilor și a condițiilor de mediu

Parametrii de intrare care influențează direct comportamentul sunt coeficienții a, b, c . Condiția de mediu relevantă (starea internă rezultată în urma procesării datelor valide) este discriminantul Δ .

Pasul 3: Găsirea categoriilor

Proprietățile majore care definesc și separă fluxurile de execuție ale programului sunt:

- C_1 (*Validitatea tipului de date*): verifică apartenența $a, b, c \in \mathbb{R}$;
- C_2 (*Valoarea coeficientului dominant*): impune restricția $a \neq 0$;
- C_3 (*Natura rădăcinilor*): dictată de semnul lui Δ .

Pașii 4 și 5: Partiționarea în alternative și scrierea specificației de testare

Pentru a eficientiza testarea și a evita generarea de combinații redundante, asociem alternativelor constrângeri. Proprietatea `[error]` va opri combinarea ulterioară, deoarece declanșează întreruperea execuției programului. Pentru a asigura o testare robustă, alternativele integrează și valorile de frontieră identificate anterior. Să detaliem:

• Parametrul a :

1. $a \in \mathbb{R}^c$ `[error];`
2. $a = 0$ `[error];`
3. $a \rightarrow 0^-$ `[property Valid_A];`
4. $a \rightarrow 0^+$ `[property Valid_A];`
5. $a \in \mathbb{R} \setminus [-\epsilon, \epsilon]$ `[property Valid_A].`

• Parametrul b :

1. $b \in \mathbb{R}^c$ `[error];`
2. $b \in \mathbb{R}$ `[property Valid_B].`

• Parametrul c :

1. $c \in \mathbb{R}^c$ `[error];`
2. $c \in \mathbb{R}$ `[property Valid_C].`

• Semnul lui Δ :

1. $\Delta \ll 0$ `[if Valid_A and Valid_B and Valid_C];`
2. $\Delta \rightarrow 0^-$ `[if Valid_A and Valid_B and Valid_C];`
3. $\Delta = 0$ `[if Valid_A and Valid_B and Valid_C];`
4. $\Delta \rightarrow 0^+$ `[if Valid_A and Valid_B and Valid_C];`
5. $\Delta \gg 0$ `[if Valid_A and Valid_B and Valid_C].`

Pașii 6 și 7: Crearea cazurilor și a datelor de test

În absența constrângerilor, simplul produs cartezian al alternativelor ar fi generat $5 \times 2 \times 2 \times 5 = 100$ de combinații. Aplicând condițiile de selecție, excludem evaluarea lui Δ pentru intrările care declanșează erori. De asemenea, pentru optimizare, într-un caz de testare la limita frontierelor, testăm doar câte o singură frontieră, reducând astfel spațiul teoretic la 11 scenarii de testare distincte, care pot fi observate în Tabelul 2.3.

Acest set de teste este optim, deoarece cuprinde:

- **4 cazuri de excepție:** acoperă validarea datelor (3 teste) și restricția matematică esențială (1 test);
- **2 cazuri pentru frontiera intrărilor:** testează stabilitatea programului când a tinde la 0 din ambele direcții;
- **2 cazuri pentru frontiera stărilor interne:** testează sensibilitatea decizională la granița dintre soluțiile reale și cele complexe;
- **3 cazuri de clasă normală:** validează comportamentul standard pentru cele trei tipuri de rădăcini.

ID Test	Combinații	a	b	c	Rezultat așteptat
TC_PC_01	a_1	True	2	1	E_1 (Excepție: tipuri invalide)
TC_PC_02	$a_5 \wedge b_1$	1	None	1	E_1 (Excepție: tipuri invalide)
TC_PC_03	$a_5 \wedge b_2 \wedge c_1$	1	2	[]	E_1 (Excepție: tipuri invalide)
TC_PC_04	$a_2 \wedge b_2 \wedge c_2$	0	2	1	E_2 (Excepție: a nu poate fi 0)
TC_PC_05	$a_3 \wedge b_2 \wedge c_2 \wedge \Delta_5$	-0.001	2	1	S_3 ($a \rightarrow 0^-$; $\Delta > 0$)
TC_PC_06	$a_4 \wedge b_2 \wedge c_2 \wedge \Delta_5$	0.001	2	1	S_3 ($a \rightarrow 0^+$; $\Delta > 0$)
TC_PC_07	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_2$	1	1.999	1	S_1 (a normal; $\Delta \rightarrow 0^-$)
TC_PC_08	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_3$	1	2	1	S_2 (a normal; $\Delta = 0$)
TC_PC_09	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_4$	1	2.001	1	S_3 (a normal; $\Delta \rightarrow 0^+$)
TC_PC_10	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_1$	1	1	1	S_1 (valori normale; $\Delta < 0$)
TC_PC_11	$a_5 \wedge b_2 \wedge c_2 \wedge \Delta_5$	1	5	6	S_3 (valori normale; $\Delta > 0$)

Tabela 2.3: Cazuri de testare pentru metoda PC

Pentru a încheia secțiunea, prezentăm implementarea automatizată a scenariilor summarize mai sus în fragmentul de cod următor.

```

1 class TestEquationSolverPC(unittest.TestCase):
2     # TC_PC_01 -> TC_PC_03: Erori de validare a tipului de date pentru
    coeficientii a, b, c
3     def test_tc_pc_01_a_invalid(self):
4         solver = EquationSolver(True, 2, 1)
5         with self.assertRaises(ValueError):
6             solver.solve()
7
8     def test_tc_pc_02_b_invalid(self):
9         solver = EquationSolver(1, None, 1)
10        with self.assertRaises(ValueError):
11            solver.solve()
12

```

```

13 def test_tc_pc_03_c_invalid(self):
14     solver = EquationSolver(1, 2, [])
15     with self.assertRaises(ValueError):
16         solver.solve()
17
18 # TC_PC_04: Valoare invalida matematic (a = 0)
19 def test_tc_pc_04_a_zero(self):
20     solver = EquationSolver(0, 2, 1)
21     with self.assertRaises(ValueError):
22         solver.solve()
23
24 # TC_PC_05: Limita stanga pentru coeficientul a (a -> 0-)
25 def test_tc_pc_05_a_limita_negativa(self):
26     solver = EquationSolver(-0.001, 2, 1)
27     x1, x2 = solver.solve()
28
29     self.assertIsInstance(x1, float)
30     self.assertIsInstance(x2, float)
31
32     self.assertAlmostEqual(max(x1, x2), 2000.49988, places=3)
33     self.assertAlmostEqual(min(x1, x2), -0.49987, places=3)
34
35 # TC_PC_06: Limita dreapta pentru coeficientul a (a -> 0+)
36 def test_tc_pc_06_a_limita_pozitiva(self):
37     solver = EquationSolver(0.001, 2, 1)
38     x1, x2 = solver.solve()
39
40     self.assertIsInstance(x1, float)
41     self.assertIsInstance(x2, float)
42
43     self.assertAlmostEqual(max(x1, x2), -0.50012, places=3)
44     self.assertAlmostEqual(min(x1, x2), -1999.49987, places=3)
45
46 # TC_PC_07: Limita stanga pentru Delta (Delta -> 0-)
47 def test_tc_pc_07_delta_limita_negativa(self):
48     solver = EquationSolver(1, 1.999, 1)
49     x1, x2 = solver.solve()
50
51     self.assertIsInstance(x1, complex)
52     self.assertIsInstance(x2, complex)
53
54     self.assertAlmostEqual(x1.real, -0.9995, places=3)
55     self.assertAlmostEqual(x2.real, -0.9995, places=3)
56
57     self.assertAlmostEqual(max(x1.imag, x2.imag), 0.03161, places=3)
58     self.assertAlmostEqual(min(x1.imag, x2.imag), -0.03161, places=3)
59
60 # TC_PC_08: Delta = 0 (0 radacina reala dubla)
61 def test_tc_pc_08_delta_zero(self):
62     solver = EquationSolver(1, 2, 1)
63     x = solver.solve()
64
65     self.assertIsInstance(x, float)
66     self.assertEqual(x, -1.0)
67
68 # TC_PC_09: Limita dreapta pentru Delta (Delta -> 0+)
69 def test_tc_pc_09_delta_limita_pozitiva(self):
70     solver = EquationSolver(1, 2.001, 1)

```

```

71     x1, x2 = solver.solve()
72
73     self.assertIsInstance(x1, float)
74     self.assertIsInstance(x2, float)
75
76     self.assertAlmostEqual(max(x1, x2), -0.96887, places=3)
77     self.assertAlmostEqual(min(x1, x2), -1.03213, places=3)
78
79     # TC_PC_10: Delta < 0 (Doua radacini complexe)
80     def test_tc_pc_10_delta_negativ(self):
81         solver = EquationSolver(1, 1, 1)
82         x1, x2 = solver.solve()
83
84         self.assertIsInstance(x1, complex)
85         self.assertIsInstance(x2, complex)
86
87         self.assertAlmostEqual(x1.real, -0.5)
88         self.assertAlmostEqual(x2.real, -0.5)
89
90         self.assertAlmostEqual(max(x1.imag, x2.imag), 0.866025, places=5)
91         self.assertAlmostEqual(min(x1.imag, x2.imag), -0.866025, places=5)
92
93     # TC_PC_11: Delta > 0 (Doua radacini reale distincte)
94     def test_tc_pc_11_delta_pozitiv(self):
95         solver = EquationSolver(1, 5, 6)
96         x1, x2 = solver.solve()
97
98         self.assertIsInstance(x1, float)
99         self.assertIsInstance(x2, float)
100
101         self.assertAlmostEqual(max(x1, x2), -2.0, places=5)
102         self.assertAlmostEqual(min(x1, x2), -3.0, places=5)

```

Secvență de cod 2.3: Fișierul `test_equation_solver.py` ilustrând clasa `TestEquationSolverPC`

Rezultatul obținut la rularea tuturor testelor din acest capitol este prezentat mai jos:

```

"D:\FMI\ANUL III\TESTAREA SISTEMELOR SOFTWARE\Proiect TSS\.venv\Scripts\
python.exe" "D:/SOFTURI/PyCharm 2025.3.3/plugins/python-ce/helpers/
pycharm/_jb_unittest_runner.py" --path "D:\FMI\ANUL III\TESTAREA
SISTEMELOR SOFTWARE\Proiect TSS\test_equation_solver.py"
Testing started at 17:02 ...
Launching unittests with arguments python -m unittest D:\FMI\ANUL III\
TESTAREA SISTEMELOR SOFTWARE\Proiect TSS\test_equation_solver.py --quiet
in D:\FMI\ANUL III\TESTAREA SISTEMELOR SOFTWARE\Proiect TSS

Ran 22 tests in 0.024s

OK

Process finished with exit code 0

```

2.3.1 Graful cauză-efect

Pentru a evidenția relația dintre condițiile de intrare și rezultatele posibile ale programului, putem defini un set de cauze și efecte asociate metodei `solve()`.

Cauzele identificate sunt:

- C1: cel puțin un coeficient nu este numeric;
- C2: coeficientul a este egal cu 0;
- C3: discriminantul este negativ, $\Delta < 0$;
- C4: discriminantul este egal cu 0, $\Delta = 0$;
- C5: discriminantul este pozitiv, $\Delta > 0$.

Efectele corespunzătoare sunt:

- E1: se generează `ValueError` pentru tipuri invalide;
- E2: se generează `ValueError` pentru $a = 0$;
- E3: se returnează două rădăcini complexe conjugate;
- E4: se returnează o rădăcină reală dublă;
- E5: se returnează două rădăcini reale distincte.

Relațiile cauză-efect pot fi sintetizate astfel:

$$C1 \Rightarrow E1$$

$$\neg C1 \wedge C2 \Rightarrow E2$$

$$\neg C1 \wedge \neg C2 \wedge C3 \Rightarrow E3$$

$$\neg C1 \wedge \neg C2 \wedge C4 \Rightarrow E4$$

$$\neg C1 \wedge \neg C2 \wedge C5 \Rightarrow E5$$

Această analiză confirmă faptul că fiecare rezultat posibil al metodei este determinat de o combinație clară de condiții de intrare și condiții interne.

Pentru a transforma analiza cauză-efect într-o formă mai apropiată de metodologia prezentată la curs, construim și un tabel de decizie. Acesta arată explicit ce combinații de cauze conduc la fiecare efect observabil al metodei `solve()`.

Notăm cu T faptul că o cauză este adevărată, cu F faptul că este falsă, iar cu “–” faptul că valoarea cauzei nu mai este relevantă, deoarece execuția programului s-a oprit anterior printr-o excepție.

Cauză / Efect	R1	R2	R3	R4	R5
C_1 : coeficient invalid	T	F	F	F	F
C_2 : $a = 0$	–	T	F	F	F
C_3 : $\Delta < 0$	–	–	T	F	F
C_4 : $\Delta = 0$	–	–	F	T	F
C_5 : $\Delta > 0$	–	–	F	F	T
E_1 : <code>ValueError</code> pentru tip invalid	X				
E_2 : <code>ValueError</code> pentru $a = 0$		X			
E_3 : două rădăcini complexe			X		
E_4 : rădăcină reală dublă				X	
E_5 : două rădăcini reale distincte					X

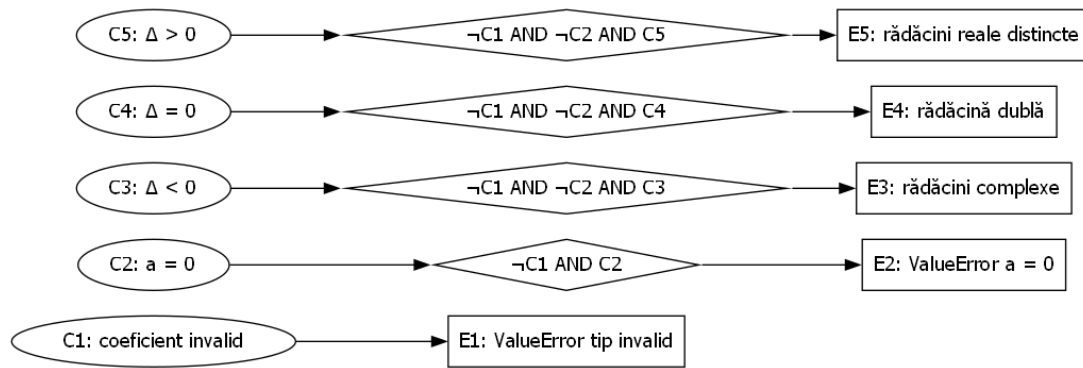
Tabela 2.4: Tabel de decizie pentru graful cauză-efect al metodei `solve()`

Regulile din tabel corespund celor cinci drumuri funcționale principale ale metodei: validare eșuată a tipului de date, încălcarea restricției $a \neq 0$, discriminant negativ, discriminant nul și discriminant pozitiv. Astfel, tabelul de decizie confirmă faptul că fiecare efect observabil este produs de o combinație unică și neambiguă de cauze.

```

1 from graphviz import Digraph
2
3 g = Digraph("CauseEffectGraph", format="png")
4 g.attr(rankdir="LR")
5
6 # Cauze
7 g.node("C1", "C1: coeficient invalid", shape="ellipse")
8 g.node("C2", "C2: a = 0", shape="ellipse")
9 g.node("C3", "C3: delta < 0", shape="ellipse")
10 g.node("C4", "C4: delta = 0", shape="ellipse")
11 g.node("C5", "C5: delta > 0", shape="ellipse")
12
13 # Noduri logice
14 g.node("N1", "!C1 && C2", shape="diamond")
15 g.node("N2", "!C1 && !C2 && C3", shape="diamond")
16 g.node("N3", "!C1 && !C2 && C4", shape="diamond")
17 g.node("N4", "!C1 && !C2 && C5", shape="diamond")
18
19 # Efecte
20 g.node("E1", "E1: ValueError tip invalid", shape="box")
21 g.node("E2", "E2: ValueError a = 0", shape="box")
22 g.node("E3", "E3: radacini complexe", shape="box")
23 g.node("E4", "E4: radacina dubla", shape="box")
24 g.node("E5", "E5: radacini reale distincte", shape="box")
25
26 # Legaturi
27 g.edge("C1", "E1")
28
29 g.edge("C2", "N1")
30 g.edge("N1", "E2")
31
32 g.edge("C3", "N2")
33 g.edge("N2", "E3")
34
35 g.edge("C4", "N3")
36 g.edge("N3", "E4")
37
38 g.edge("C5", "N4")
39 g.edge("N4", "E5")
40
41 g.render("cause_effect_graph", view=True)

```

Figura 2.1: Graful cauză-efect pentru metoda `solve()`

Capitolul 3

Testare structurală

Testarea structurală (White-Box Testing) reprezintă o metodă de testare care analizează structura internă a codului sursă. Aceasta urmărește fluxul de control, ramificațiile și condițiile logice ale programului, având ca scop executarea tuturor instrucțiunilor și deciziilor cel puțin o dată în timpul testării.

În cadrul acestui capitol se realizează analiza structurală a metodei `solve()` din clasa `EquationSolver`, prin construirea grafului de flux de control (Control Flow Graph), determinarea complexității ciclomatice, identificarea drumurilor independente și evaluarea gradului de acoperire a codului pe baza cazurilor de test definite anterior.

3.1 Graful de flux de control (CFG)

Graful de flux de control (CFG) reprezintă o modelare grafică a modului în care instrucțiunile unui program sunt executate. Nodurile grafului corespund blocurilor de instrucțiuni, iar muchiile indică tranzițiile dintre acestea.

Analiza CFG permite evidențierea ramificațiilor logice și identificarea drumurilor de execuție ale programului.

Pentru metoda `solve()`, graful de flux de control surprinde etapele principale ale algoritmului: validarea coeficienților, verificarea condiției $a = 0$, calculul discriminantului și ramificarea execuției în funcție de valoarea acestuia.

3.1.1 Generarea automată a CFG-ului

Pentru automatizarea procesului de construire a grafului de flux de control, a fost utilizată biblioteca `Graphviz`. Codul de mai jos definește nodurile și muchiile corespunzătoare fluxului de execuție al metodei `solve()`.

```
1 from graphviz import Digraph
2
3 cfg = Digraph("CFG")
4
5 # Noduri
6 cfg.node("1", "Start")
7 cfg.node("2", "Intrari valide?")
8 cfg.node("3", "ValueError")
9 cfg.node("4", "a == 0?")
10 cfg.node("5", "ValueError")
11 cfg.node("6", " $\delta = b^2 - 4ac$ ")
12 cfg.node("7", " $\delta < 0?$ ")
13 cfg.node("8", "Radacini complexe")
14 cfg.node("9", " $\delta = 0?$ ")
15 cfg.node("10", "Radacina dubla")
16 cfg.node("11", "Doua radacini reale")
17 cfg.node("12", "Stop")
18
19 # Muchii
20 cfg.edge("1", "2")
21
22 cfg.edge("2", "3", label="Nu")
23 cfg.edge("2", "4", label="Da")
24 cfg.edge("3", "12")
25
26 cfg.edge("4", "5", label="Da")
27 cfg.edge("4", "6", label="Nu")
28 cfg.edge("5", "12")
29
30 cfg.edge("6", "7")
31
32 cfg.edge("7", "8", label="Da")
33 cfg.edge("7", "9", label="Nu")
34 cfg.edge("8", "12")
35
36 cfg.edge("9", "10", label="Da")
37 cfg.edge("9", "11", label="Nu")
38
39 cfg.edge("10", "12")
40 cfg.edge("11", "12")
41
42 # Genereaza imaginea
43 cfg.render("equation_solver_cfg", format="png", cleanup=True)
44
45 print("CFG generated successfully!")
```

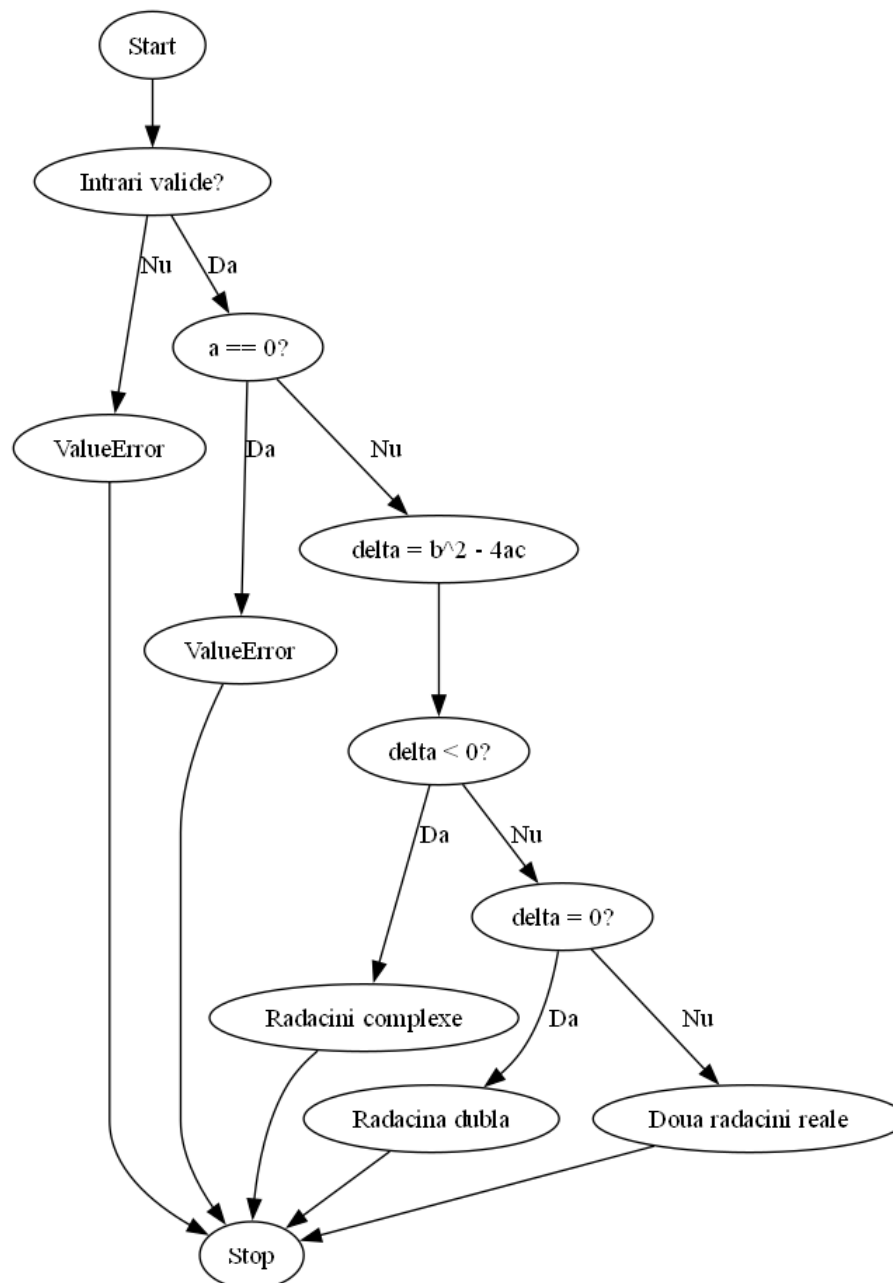


Figura 3.1: Graful de flux de control generat automat folosind Graphviz pentru metoda `solve()`

Graful evidențiază succesiunea pașilor de execuție. Execuția începe în nodul *Start*, unde sunt validate intrările. Dacă cel puțin un coeficient nu este numeric, execuția este întreruptă prin generarea unei excepții.

În cazul în care datele sunt valide, se verifică valoarea coeficientului a . Dacă $a = 0$, execuția este oprită, deoarece ecuația nu mai este de gradul al doilea. În caz contrar, se calculează discriminantul $\Delta = b^2 - 4ac$.

În funcție de valoarea discriminantului, execuția se ramifică în trei situații:

- $\Delta < 0$ – două rădăcini complexe;

- $\Delta = 0$ – o rădăcină reală dublă;
- $\Delta > 0$ – două rădăcini reale distincte.

După determinarea rezultatului, execuția ajunge în nodul final *Stop*.

3.2 Analiza acoperirii (Coverage)

Analiza acoperirii codului urmărește în ce măsură cazurile de test exercită diferitele componente ale programului. Scopul este verificarea faptului că toate instrucțiunile și toate ramurile logice sunt executate cel puțin o dată.

Acoperirea la nivel de decizie este strâns corelată cu acoperirea la nivel de ramură (branch coverage), deoarece fiecare decizie produce ramuri distincte de execuție. Pentru metoda `solve()`, sunt analizate două tipuri principale de acoperire: acoperirea la nivel de instrucțiuni și acoperirea la nivel de decizie.

3.2.1 Acoperire la nivel de instrucțiuni

Acoperirea la nivel de instrucțiuni (Statement Coverage) presupune executarea fiecărei instrucțiuni cel puțin o dată.

În cazul metodei `solve()`, acest lucru implică testarea tuturor situațiilor posibile:

- intrări invalide (coeficienți nenumerici);
- cazul invalid în care $a = 0$;
- $\Delta < 0$;
- $\Delta = 0$;
- $\Delta > 0$.

Test Case	Input	Rezultat	Instrucțiuni/Noduri parcurse
TC_CE_01	(a="x", b=2, c=1)	ValueError	Start → validare input → eroare
TC_CE_02	(a=0, b=2, c=1)	ValueError	Start → validare → $a = 0$ → eroare
TC_CE_03	(1,1,1)	rădăcini complexe	Start → $\Delta < 0$
TC_CE_04	(1,2,1)	rădăcină dublă	Start → $\Delta = 0$
TC_CE_05	(1,5,6)	două rădăcini reale	Start → $\Delta > 0$

Tabela 3.1: Statement coverage pentru metoda `solve()`

Acoperirea la nivel de decizie presupune evaluarea fiecărei condiții logice atât cu valoarea *true*, cât și *false*.

Pentru metoda analizată, aceasta implică:

- verificarea validității intrărilor;

- verificarea condiției $a = 0$;
- evaluarea celor trei cazuri determinate de discriminant.

Prin cazurile de test definite în Capitolul 2, toate ramurile logice sunt exercitate, asigurând acoperire completă la nivel de instrucțiuni și decizie.

Decizie	True	False
Intrări valide?	TC_CE_02, TC_CE_03, TC_CE_04, TC_CE_05	TC_CE_01
$a == 0$	TC_CE_02	TC_CE_03, TC_CE_04, TC_CE_05
$\Delta < 0$	TC_CE_03	TC_CE_04, TC_CE_05
$\Delta = 0$	TC_CE_04	TC_CE_03, TC_CE_05

Tabela 3.2: Decision/Branch coverage pentru metoda solve()

3.2.2 Legătura dintre cazurile de test și acoperirea codului

Pentru a demonstra acoperirea completă, corelăm cazurile de test definite cu ramurile din CFG.

- **Validarea tipului de date (E_1)**
acoperită de: TC_CE_01, TC_PC_01, TC_PC_02, TC_PC_03
- **Restricția $a = 0$ (E_2)**
acoperită de: TC_CE_02, TC_PC_04
- $\Delta < 0$
acoperit de: TC_CE_03, TC_PC_10, TC_AVF_03
- $\Delta = 0$
acoperit de: TC_CE_04, TC_PC_08
- $\Delta > 0$
acoperit de: TC_CE_05, TC_PC_11, TC_AVF_01, TC_AVF_02, TC_AVF_04

3.3 Rularea instrumentului coverage.py

Pentru a valida practic gradul de acoperire al codului, am utilizat instrumentul `coverage.py`. Acesta permite monitorizarea liniilor de cod executate în timpul rulării testelor automate și generează un raport care evidențiază numărul de instrucțiuni executate, instrucțiunile neacoperite și procentul total de acoperire.

Înainte de rularea testelor, am șters rezultatele anterioare de coverage folosind comanda `coverage erase`. Ulterior, testele au fost executate prin framework-ul `unittest`, iar raportul final a fost generat cu opțiunea `-m`, care afișează și liniile lipsă, dacă acestea există.

Comanda utilizată a fost:

```
coverage erase
coverage run --branch -m unittest test_equation_solver.py
coverage report -m
```

Rezultatul obținut în consolă a fost următorul:

```
Ran 22 tests in 0.007s
```

```
OK
```

Name	Stmts	Miss	Branch	BrPart	Cover	Missing
-----	-----	-----	-----	-----	-----	-----
equation_solver.py	23	0	8	0	100%	
test_equation_solver.py	139	0	0	0	100%	
-----	-----	-----	-----	-----	-----	-----
TOTAL	162	0	8	0	100%	

După cum se observă, fișierul *equation_solver.py* are o acoperire de **100%**, ceea ce înseamnă că toate instrucțiunile din implementarea clasei `Equation Solver` au fost executate cel puțin o dată de cazurile de test definite.

De asemenea, fișierul *test_equation_solver.py* are tot o acoperire de **100%**, iar raportul total indică 162 de instrucțiuni executate și 0 instrucțiuni neacoperite. Acest rezultat confirmă faptul că setul de teste propus acoperă complet implementarea analizată, incluzând atât cazurile de succes, cât și cazurile de eroare. Prin această corespondență se confirmă că toate ramurile logice sunt testate, iar acoperirea codului este completă.

3.4 Acoperire la nivel de condiție

Acoperirea la nivel de condiție (Condition Coverage) urmărește evaluarea fiecărei condiții elementare dintr-o expresie logică atât cu valoarea adevărat, cât și fals.

Scopul acestei metrice este de a verifica toate componentele unei condiții, nu doar rezultatul final al deciziei.

În cazul metodei `solve()`, condițiile sunt în mare parte simple (nu există expresii logice compuse explicite), astfel încât acoperirea la nivel de condiție coincide în mare măsură cu acoperirea la nivel de decizie.

Condițiile identificate în cod sunt:

- validarea tipului coeficienților: `not all(...)`
- verificarea coeficientului dominant: `a == 0`
- evaluarea discriminantului: `delta < 0`
- verificarea cazului limită: `math.isclose(delta, 0)`

Pentru a demonstra acoperirea la nivel de condiție, analizăm pentru fiecare condiție cazurile în care aceasta este evaluată atât ca adevărat, cât și fals.

Condiție	Evaluare True	Evaluare False
<code>not all(...)</code>	TC_CE_01	TC_CE_02–05, TC_PC_04–11, TC_AVF_01–04
<code>a == 0</code>	TC_CE_02, TC_PC_04	toate testele valide cu $a \neq 0$
<code>delta < 0</code>	TC_CE_03, TC_PC_10, TC_AVF_03	TC_CE_04–05, TC_PC_08–09, TC_AVF_01–02–04
<code>isclose(delta, 0)</code>	TC_CE_04, TC_PC_08	cazurile cu $\Delta < 0$ și $\Delta > 0$

Tabela 3.3: Acoperirea condițiilor pentru metoda `solve()`

Se observă că fiecare condiție este evaluată atât cu valoarea adevărat, cât și cu valoarea fals, ceea ce confirmă obținerea unei acoperiri complete la nivel de condiție.

3.4.1 Condition/Decision Coverage

Acoperirea condiție/decizie (Condition/Decision Coverage) combină acoperirea la nivel de condiție cu acoperirea la nivel de decizie, impunând atât evaluarea condițiilor individuale, cât și a rezultatului global al deciziei.

Această metrică oferă o analiză mai riguroasă decât acoperirea la nivel de decizie simplă.

Pentru metoda `solve()`, această acoperire este satisfăcută, deoarece fiecare condiție este evaluată pentru ambele rezultate posibile, iar fiecare ramură decizională este parcursă de cel puțin un caz de test.

3.4.2 Multiple Condition Coverage

Acoperirea la nivel de condiții multiple (Multiple Condition Coverage) presupune testarea tuturor combinațiilor posibile de valori de adevăr ale condițiilor individuale dintr-o expresie logică.

Această metodă este mai strictă decât acoperirea condiție/decizie, însă poate conduce la un număr foarte mare de cazuri de test.

În cazul metodei `solve()`, nu există condiții compuse explicite (de tip `c1 && c2` sau `c1 || c2`), astfel încât această metrică nu aduce informații suplimentare relevante.

3.4.3 MC/DC

Acoperirea MC/DC (Modified Condition/Decision Coverage) impune ca fiecare condiție individuală dintr-o decizie să ia atât valoarea **True**, cât și valoarea **False**, ca fiecare decizie să fie evaluată atât cu rezultat adevărat, cât și cu rezultat fals, și ca fiecare condiție individuală să influențeze în mod independent rezultatul deciziei.

Această metodă este mai puternică decât acoperirea condiție/decizie simplă, deoarece nu verifică doar evaluarea condițiilor, ci și efectul fiecărei condiții asupra rezultatului global al deciziei. În același timp, MC/DC evită explozia combinatorică specifică acoperirii condițiilor multiple.

Pentru metoda analizată, MC/DC este relevantă în special pentru prima decizie din metoda `solve()`, unde se realizează validarea coeficienților:

```
if not all(isinstance(x, (int, float)) and not isinstance(x, bool)
           for x in [self.a, self.b, self.c])
```

Această decizie conține o condiție compusă. Pentru fiecare coeficient x , validarea poate fi exprimată astfel:

$$V(x) = C_1(x) \wedge C_2(x)$$

unde:

$$C_1(x) = \text{isinstance}(x, (\text{int}, \text{float}))$$

și

$$C_2(x) = \text{not isinstance}(x, \text{bool})$$

Astfel, condiția C_1 verifică dacă valoarea este de tip numeric, iar condiția C_2 verifică dacă valoarea nu este de tip boolean. Această a doua verificare este necesară deoarece, în Python, tipul `bool` este subclasă a tipului `int`, iar valorile `True` și `False` ar fi acceptate de o verificare simplă cu `isinstance(x, (int, float))`.

Pentru a evidenția principiul MC/DC, putem analiza validarea unui singur coeficient, păstrând ceilalți coeficienți valizi. De exemplu:

$$V(x) = C_1 \wedge C_2$$

Test	Valoare x	C_1	C_2	$V(x)$
t1	2	True	True	True
t2	"text"	False	True	False
t3	True	True	False	False

Perechea **t1–t2** demonstrează influența independentă a condiției C_1 , deoarece C_2 rămâne adevărată, iar rezultatul validării se schimbă din adevărat în fals. Perechea **t1–t3** demonstrează influența independentă a condiției C_2 , deoarece C_1 rămâne adevărată, iar rezultatul validării se schimbă din adevărat în fals.

Prin urmare, pentru prima decizie din metoda `solve()`, MC/DC poate fi aplicată și este acoperită prin teste care includ coeficienți numerici valizi, coeficienți nenumerici și valori de tip boolean. Pentru celelalte decizii din metodă, precum `self.a == 0`, `math.isclose(delta, 0.0, abs_tol=1e-9)` și `delta < 0`, condițiile sunt simple, astfel încât MC/DC nu aduce informații suplimentare semnificative față de acoperirea la nivel de decizie.

3.4.4 Path Coverage

Testarea la nivel de cale (Path Coverage) presupune generarea unor cazuri de test astfel încât fiecare cale posibilă din graful de flux de control să fie parcursă.

În cazul metodei `solve()`, căile de execuție sunt finite și corespund ramurilor principale:

- eroare de tip (E_1)
- eroare pentru $a = 0$ (E_2)
- $\Delta < 0$
- $\Delta = 0$
- $\Delta > 0$

Acestea sunt deja acoperite prin cazurile de test definite în Capitolul 2, ceea ce asigură o acoperire completă la nivel de cale.

3.4.5 LCSAJ

Acoperirea LCSAJ (Linear Code Sequence and Jump) analizează secvențele liniare de cod urmate de salturi ale fluxului de execuție.

Această metodă oferă o granularitate mai mare decât acoperirea la nivel de decizie.

Pentru metoda `solve()`, structura simplă a codului și numărul redus de ramuri nu justifică aplicarea acestei metode, deoarece nu aduce beneficii suplimentare semnificative.

3.5 Complexitatea ciclomatică (McCabe)

Complexitatea ciclomatică reprezintă o măsură a complexității logice a unui program și indică numărul minim de drumuri independente care trebuie testate pentru a asigura acoperirea completă a codului. Aceasta a fost introdusă de McCabe și se calculează folosind relația:

$$V(G) = E - N + 2P$$

unde:

- E reprezintă numărul de muchii ale grafului de flux de control;
- N reprezintă numărul de noduri ale grafului;
- P reprezintă numărul de componente conexe ale grafului (în cazul analizat, $P = 1$).

Complexitatea ciclomatică poate fi determinată și prin numărul de decizii din program, folosind relația:

$$V(G) = D + 1$$

unde D reprezintă numărul de decizii.

În cazul metodei `solve()`, se identifică patru decizii principale:

- verificarea validității coeficienților;
- verificarea condiției $a = 0$;
- verificarea condiției $\Delta < 0$;
- verificarea condiției $\Delta = 0$ (`math.isclose(delta, 0)`).

Astfel, complexitatea ciclomatică este:

$$V(G) = 4 + 1 = 5$$

Această valoare indică faptul că există cinci drumuri independente care trebuie testate pentru a asigura acoperirea completă a metodei.

Alternativ, folosind formula bazată pe graful de control:

$$V(G) = E - N + 2P$$

unde:

- $E = 15$ (muchii)
- $N = 12$ (noduri)
- $P = 1$ (componentele conexe)

$$V(G) = 15 - 12 + 2 = 5$$

Analiza realizată se bazează pe tehnica *Basis Path Testing*, care utilizează complexitatea ciclomatică pentru a determina setul minim de drumuri independente necesare testării complete.

3.6 Circuite independente

Circuitele independente (sau drumurile independente) reprezintă trasee distincte în graful de flux de control, fiecare introducând cel puțin o muchie nouă față de drumurile parcurse anterior

Numărul acestora este determinat de complexitatea ciclomatică. În cazul metodei `solve()`, complexitatea ciclomatică este:

$$V(G) = 5$$

Prin urmare, există cinci drumuri independente care trebuie testate.

- **Drumul 1:** validarea intrărilor eșuează, iar metoda generează `ValueError` pentru coeficienți nenumerici;
- **Drumul 2:** coeficienții sunt validați cu succes, însă $a = 0$, iar metoda generează `ValueError`;
- **Drumul 3:** coeficienții sunt validați, $a \neq 0$, iar $\Delta < 0$, rezultând două rădăcini complexe;
- **Drumul 4:** coeficienții sunt validați, $a \neq 0$, iar $\Delta = 0$, rezultând o rădăcină dublă;

- **Drumul 5:** coeficienții sunt validați, $a \neq 0$, iar $\Delta > 0$, rezultând două rădăcini reale distincte.

Testarea acestor cinci drumuri asigură parcurgerea tuturor ramurilor logice din cadrul metodei `solve()`. Rezultatele obținute confirmă faptul că metoda analizată are o structură logică relativ simplă, ceea ce facilitează procesul de testare, analiză și mentenanță ulterioară.

Drum	Cale în CFG
P1	$1 \rightarrow 2 \rightarrow 3 \rightarrow 12$
P2	$1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 12$
P3	$1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 12$
P4	$1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 10 \rightarrow 12$
P5	$1 \rightarrow 2 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 11 \rightarrow 12$

Tabela 3.4: Drumurile independente identificate în CFG